SC2002 Object-Oriented Design & Programming

Restaurant Rush

Group Assignment Report

Lab group: [LAB GROUP] Group: [GROUP NO.]
Repository: https://github.com/pyizaw/NTU_SCE1_SC2002
Video: [VIDEO LINK]

Members

[Member 1 name] - [Matric no.]
[Member 2 name] - [Matric no.]
[Member 3 name] - [Matric no.]
[Member 4 name] - [Matric no.]
[Member 5 name] - [Matric no.]

Contents

1. Overview 3
1.1 How to build and run 3
1.2 Rules we defined 3
2. UML Class Diagrams 5
2.1 Stage 1: basic restaurant operation (tag stage-1) 6
2.2 Stage 2: VIP and Critic customers (tag stage-2) 7
2.3 Stage 3: Host, ComboMeal and Happy Hour (tag stage-3, final design) 8
3. UML Sequence Diagram 9
3.1 Life cycles (supporting diagram) 12
4. Design Considerations 14
4.1 Key abstractions and responsibilities 14
4.2 Design principles reflected in the code 14
4.3 Trade-offs 15
4.4 Alternative design considered 15
4.5 How the player's choices change the result 16
5. Functional Evidence and Test Notes 17
5.1 Rule checks 17
5.2 Console screenshots 17
6. Development Process and Stage Changes 23
6.1 Stage 1 → Stage 2: 6 source files, +143 / -6 lines 23
6.2 Stage 2 → Stage 3: 11 source files, +353 / -8 lines 23
6.3 Optional window version (tag gui-extra) 24
7. Reflections 26
7.1 [Member 1 name] 26
7.2 [Member 2 name] 26
7.3 [Member 3 name] 26
7.4 [Member 4 name] 27
7.5 [Member 5 name] 27
8. Declarations and AI History 28

1. Overview

Restaurant Rush is a turn-based command-line game in Java 17 with no external libraries. The player is the manager. Every turn, the player decides what the Host, Waiter and Chef do. Customers choose their own food and the Cashier works automatically. After 18 turns the game reports **Victory** if all three targets are met, and **Defeat** otherwise:

- at least 4 customers paid;
- revenue is at least $45;
- the average satisfaction of paid customers is at least 60.

The game was built in three runnable stages, each one a single commit with its own tag. Each stage extends the saved previous stage:

- `stage-1`: basic operation;
- `stage-2`: VIP and Critic customers;
- `stage-3`: Host, ComboMeal and Happy Hour.

An optional window version is a separate commit after Stage 3 (tag `gui-extra`). It changes no rules. This report describes the final Stage 3 design and how it evolved from Stage 1.

1.1 How to build and run

- **IntelliJ IDEA:** open the repository folder, choose any JDK 17+ as the Project SDK, then run **Play (terminal)**, **Play (window)** or a **Replay_stageN_...** configuration.
- **Command line:** `javac --release 17 -d out $(find src -name "*.java")` then `java -cp out restaurantrush.Main`.
- **Replay a recorded game:** `java -cp out restaurantrush.Main runs/stage3-victory.txt`. Commands are read from the file and echoed, and lines starting with # explain each decision. Check out the matching tag first (`git checkout stage-2`).
- **Optional window:** `java -cp out restaurantrush.Main --gui` (tag `gui-extra`).

1.2 Rules we defined

The brief leaves some rules to the group. Every rule below is shown on the welcome screen and in the README.

Rule	Our definition
Food choice	Customer number n orders the menu item at position (n - 1) mod menu size. The rule is deterministic, so every run is reproducible.
Arrivals	Turn 1 Regular, turn 4 VIP, turn 7 Critic, turns 10, 13 and 16 Regular (in Stage 1 every customer is Regular).
Critic	Order preference: the item with the most preparation units (ties go to the cheaper one). Reaction to service: if the dish sat READY for more than one turn before being served, it has gone cold and the Critic loses 20 satisfaction at once.
Combo choice	Combos are ordinary menu items placed after the three dishes, so customers #4, #5 and #6 pick the three combos by the same food rule. Price = 90% of the two components, rounded to the nearest cent; units = sum of the components.
Host	Acts before the Waiter and seats at most one queued customer per turn; the Waiter can still seat customers.

Rule	Our definition
Happy Hour	Offered at the start of every turn until used (once per game). It is active on the activation turn and the next, and the remaining duration is shown. Orders taken while it is active get 20% off, applied after the VIP discount, and the price is locked in the order. At the start of each active turn, customers waiting for a table or food recover 15 satisfaction (max 100).
Seating	A customer is seated at the lowest-numbered free table.
Invalid choices	A rejected command is explained and does not use up that staff member's task; the manager is asked again.

2. UML Class Diagrams

The brief asks for one class diagram per stage, and each must match the code of its tag. To guarantee this, the members of every class (attributes and operations with visibility, static members underlined, abstract members in italics) were **generated from the source of each tag**. The relationships and notes were then added by hand. Simple `get...()` accessors are omitted to keep the diagrams readable.

All three diagrams share one colour code, and the PlantUML source and zoomable SVG versions are in `docs/diagrams/`.

Colour	Meaning
Blue	Boundary: `view`
Orange	Control: `controller`
Purple	Staff
Green	Domain
Yellow	Menu and pricing
Grey	Enums
Bright green fill (Stages 2 and 3)	Class is new in that stage
Orange fill (Stages 2 and 3)	Existing class changed in that stage

The packages follow the entity / control / boundary split:

- `view` holds the boundary classes;
- `controller` holds the control classes;
- `model`, `staff` and `enums` hold the entities and the domain logic.

The notes on the diagrams point out the main OO ideas:

- **Encapsulation.** `Customer` keeps every field private. Its state only changes through `seatAt`, `placeOrder`, `serve`, `pay` and `leave`, and each of these checks the current `CustomerState` first. `Staff.perform()` enforces "one task per turn", and a failed task does not use up the turn. `Order` locks its price when it is created.
- **Polymorphism.** The controller and `Restaurant` only hold `Customer` references, while `RegularCustomer`, `VIPCustomer` and `CriticCustomer` override `getSatisfactionLossPerTurn()`, `billFor()`, `chooseDish()` and `reactToService()`. Likewise, `GameController` calls `staff.carryOut(command, ...)` for every directed role, and each `Staff` subclass decides in `handle()` which commands it accepts and in `availableTasks()` which options it offers.
- **Composition.** `Restaurant` creates and owns its `Tables`. `GameController` creates and owns the staff, menu, Happy Hour and restaurant of one game. A `ComboMeal` is a `MenuItem` built from two existing `MenuItems` (shown as aggregation because the components also belong to the menu).
- **Inheritance:** `Staff` → `Host`, `Waiter`, `Chef`, `Cashier`; `Customer` → `RegularCustomer`, `VIPCustomer`, `CriticCustomer`; `MenuItem` → `ComboMeal`.
- **Multiplicities:**
  - 2 tables per restaurant;
  - 0..1 customer per table and 0..1 order per customer;

- exactly 1 menu item per order;
- 2 components per combo;
- 6 scheduled arrivals;
- 2 directed staff in Stages 1-2 and 3 in Stage 3 (`{ordered}`, the order in which they act).

2.1 Stage 1: basic restaurant operation (tag stage-1)

Stage 1 already contains the extension points that later stages use: the abstract `Customer`, the abstract `Staff` with its template method `carryOut()`, the `CustomerFactory` arrival schedule and the `GameView` interface. No Stage 2 or 3 class is present, as checked on the tag.

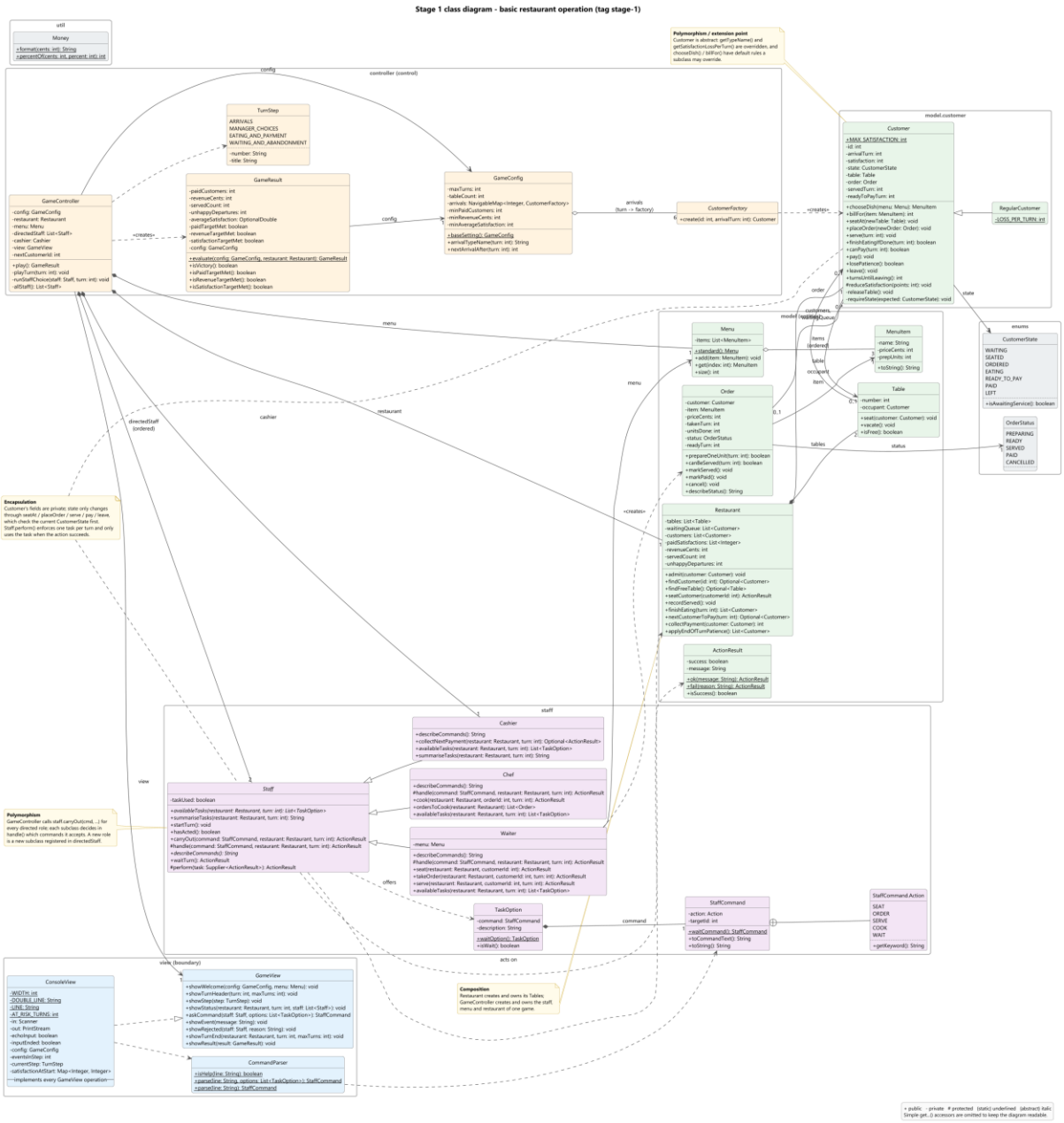


Figure 1. Stage 1 class diagram (matches tag stage-1).

2.2 Stage 2: VIP and Critic customers (tag stage-2)

The changes in this stage are colour-coded:

- **New** (bright green): `VIPCustomer` and `CriticCustomer`.
- **Changed** (orange): `Customer` (the `reactToService()` hook), `Waiter` (reports the reaction), `GameConfig` (the schedule) and `ConsoleView` (explains the customer types).
- **Untouched:** the controller, restaurant, orders, Chef and Cashier.

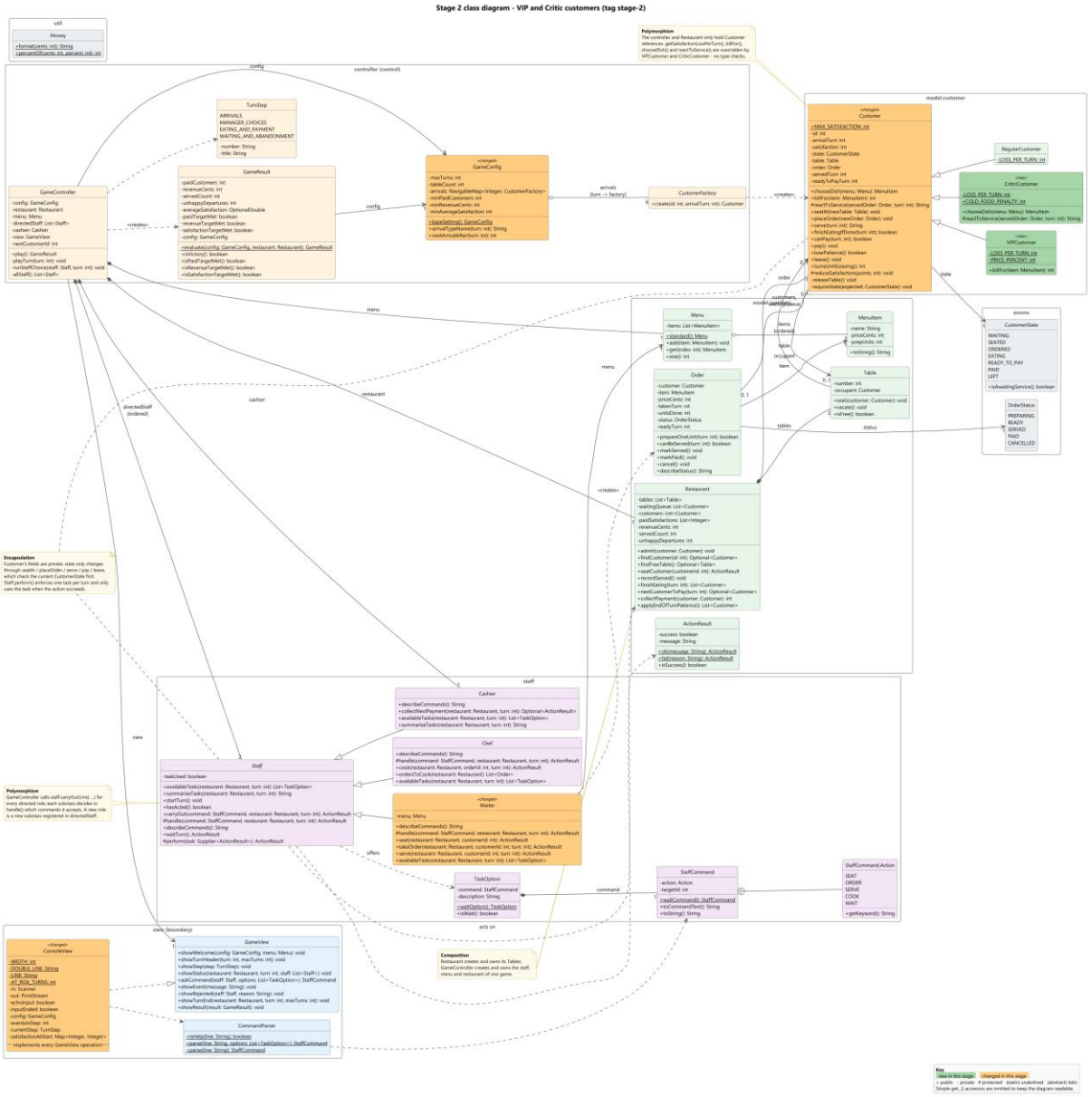


Figure 2. Stage 2 class diagram (matches tag stage-2).

2.3 Stage 3: Host, ComboMeal and Happy Hour (tag stage-3, final design)

The changes in this stage are colour-coded:

- **New:** `Host`, `ComboMeal`, `HappyHour` and `PriceCalculator`.
- **Changed:** `GameController`, `Menu`, `Restaurant`, `Customer`, `Waiter`, `GameView` and `ConsoleView`. The reason for each change is given in Section 6.

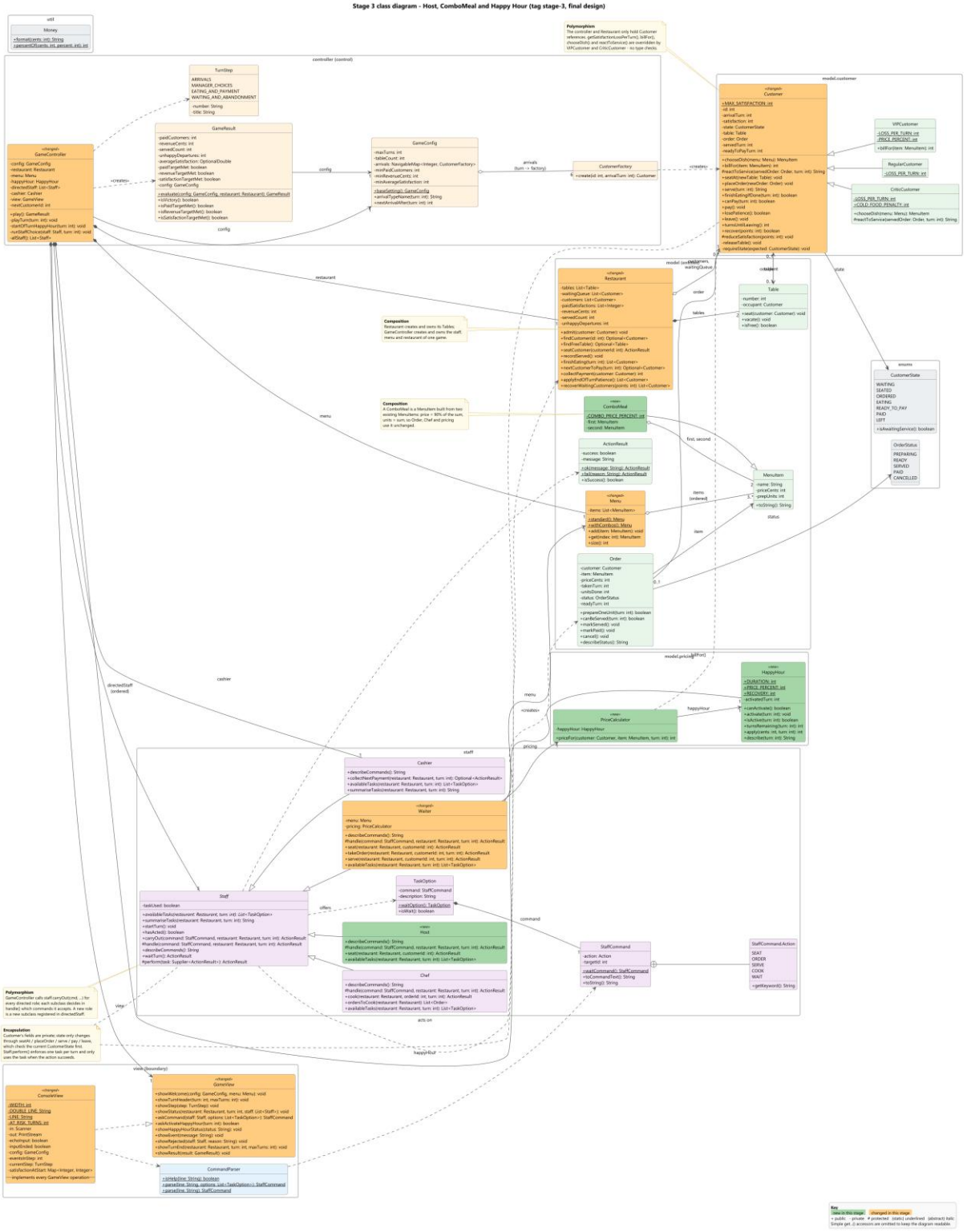


Figure 3. Stage 3 class diagram, the final design (matches tag stage-3). SwingView from gui-extra is not part of the stage and is not drawn.

3. UML Sequence Diagram

Item	Description
Use case	The manager uses Happy Hour and Waiter priority so that late customer #5 still pays before the game ends.
Stage	Stage 3 (tag `stage-3`). Reproducible with `runs/stage3-victory.txt`, turns 13 to 18.
Starting conditions	Start of turn 13: revenue $46.80, 3 paid, average satisfaction 83.7. Table 1 is free. Table 2 holds customer #4 (Regular) whose Salad + Burger Combo is already READY. Happy Hour is unused. Customer #5 (Regular) is scheduled to arrive on turn 13.
Manager choices	Turn 13: switch on Happy Hour (y); the Host seats #5; the Waiter takes #5's order instead of serving #4 (a constrained choice, since the Waiter has one task); the Chef starts #5's 3-unit combo. Turns 14-15: the Chef finishes it. Turn 16: the Waiter serves #5.
Outcome	#5 eats on turn 17 and pays $19.44 on turn 18, giving VICTORY: 5 paid, $87.84, average 86.6. If the Waiter had served #4 first on turn 13, #5's combo would only be READY on turn 16 and served on turn 17, so #5 could not pay within 18 turns: 4 paid and $68.40.

The scenario covers every element the brief asks for:

- customer arrival and order;
- Host, Waiter and Chef assignments;
- READY, then SERVED, then eating, then payment;
- the result check;
- a Stage 3 behaviour (Happy Hour pricing and recovery) with a manager choice that changes the outcome.

To stay readable, the diagram is split into two parts with continuous message numbers:

- part 1 is turn 13 (messages 1-43);
- part 2 is turns 14-18 and the result (messages 44-76).

How the interactions implement the use case

- **Loops.** All messages run inside `GameController.play()`, which loops over turns 1..18. Within a turn, the controller loops over `directedStaff = [Host, Waiter, Chef]`. Each staff prompt is itself a loop: `askCommand()` is repeated until the staff member has acted, because a rejected command (`showRejected`) does not use up the task.
- **Polymorphic dispatch.** The controller sends the same message, `carryOut(command, restaurant, turn)`, to every role. `carryOut()` is a final template method in `Staff`: it handles `wait` itself and otherwise calls the subclass's `handle()`. The Host turns it into `seat()`, the Waiter into `takeOrder()` or `serve()` and the Chef into `prepareOneUnit()`. The controller never checks which role it is talking to.
- **Pricing (Stage 3 behaviour).** The Waiter asks `PriceCalculator.priceFor()` for the bill. The calculator first asks the customer for its own price (`billFor()`, which VIPs override with 10% off) and then applies `HappyHour.apply()`. This gives 2430 → 1944 cents, which is locked into the new `Order`.
- **Guards.** `opt [happyHour.canActivate()]` offers Happy Hour; `opt [happyHour.isActive(13)]` applies recovery. `alt` frames show the valid and invalid branches of seating and serving: `canBeServed()` only allows a dish that became READY on an earlier turn (readyTurn 15 < 16). The final `alt` shows the three victory targets.
- **Food and payment flow:**

- `Order` moves PREPARING → READY (turn 15) → SERVED (turn 16).
- `finishEatingIfDone(17)` makes #5 READY_TO_PAY.
- On turn 18 the automatic Cashier asks `Restaurant.nextCustomerToPay()` (oldest first, at most one per turn) and `collectPayment()` adds the bill, records satisfaction 84, and marks the customer and order PAID.

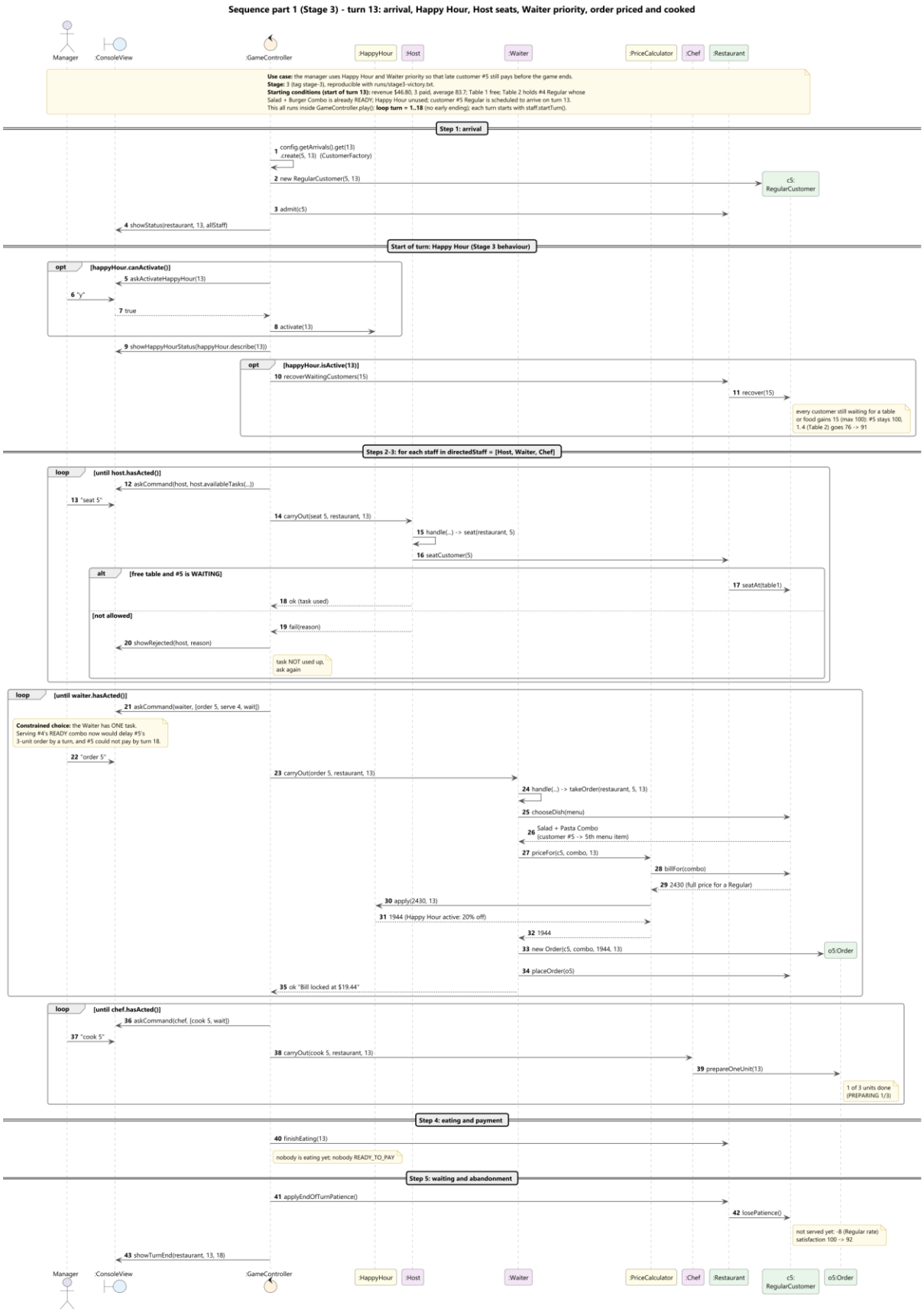


Figure 4a. Sequence part 1 - turn 13: arrival, Happy Hour, Host seats, Waiter priority, order priced and cooked.

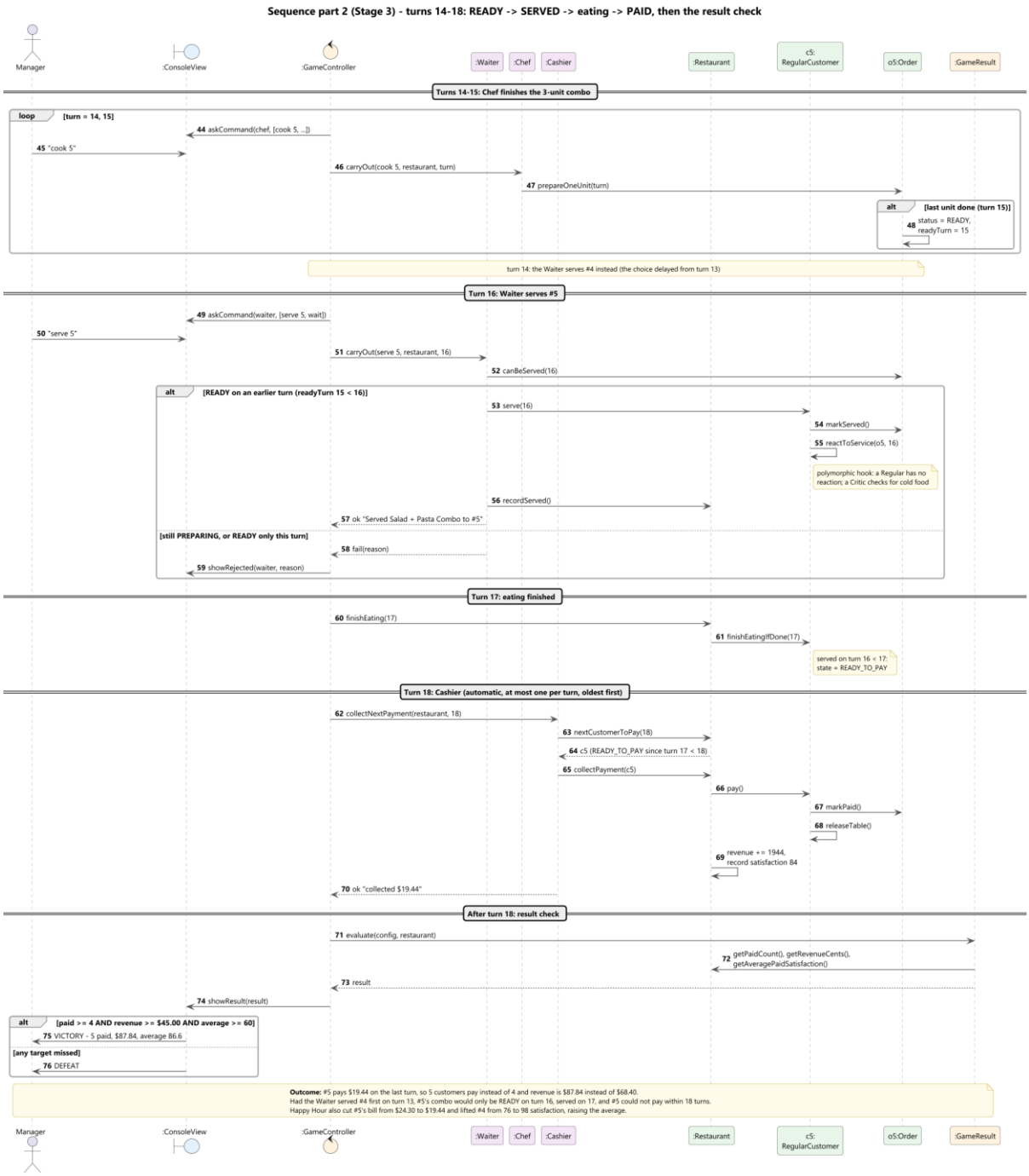


Figure 4b. Sequence part 2 - turns 14 to 18: READY, SERVED, eating, PAID, then the result check.

3.1 Life cycles (supporting diagram)

This state diagram summarises the rules that the sequence relies on: which method moves a customer or an order to its next state, and under which guard. It matches the checks inside `Customer` and `Order`.

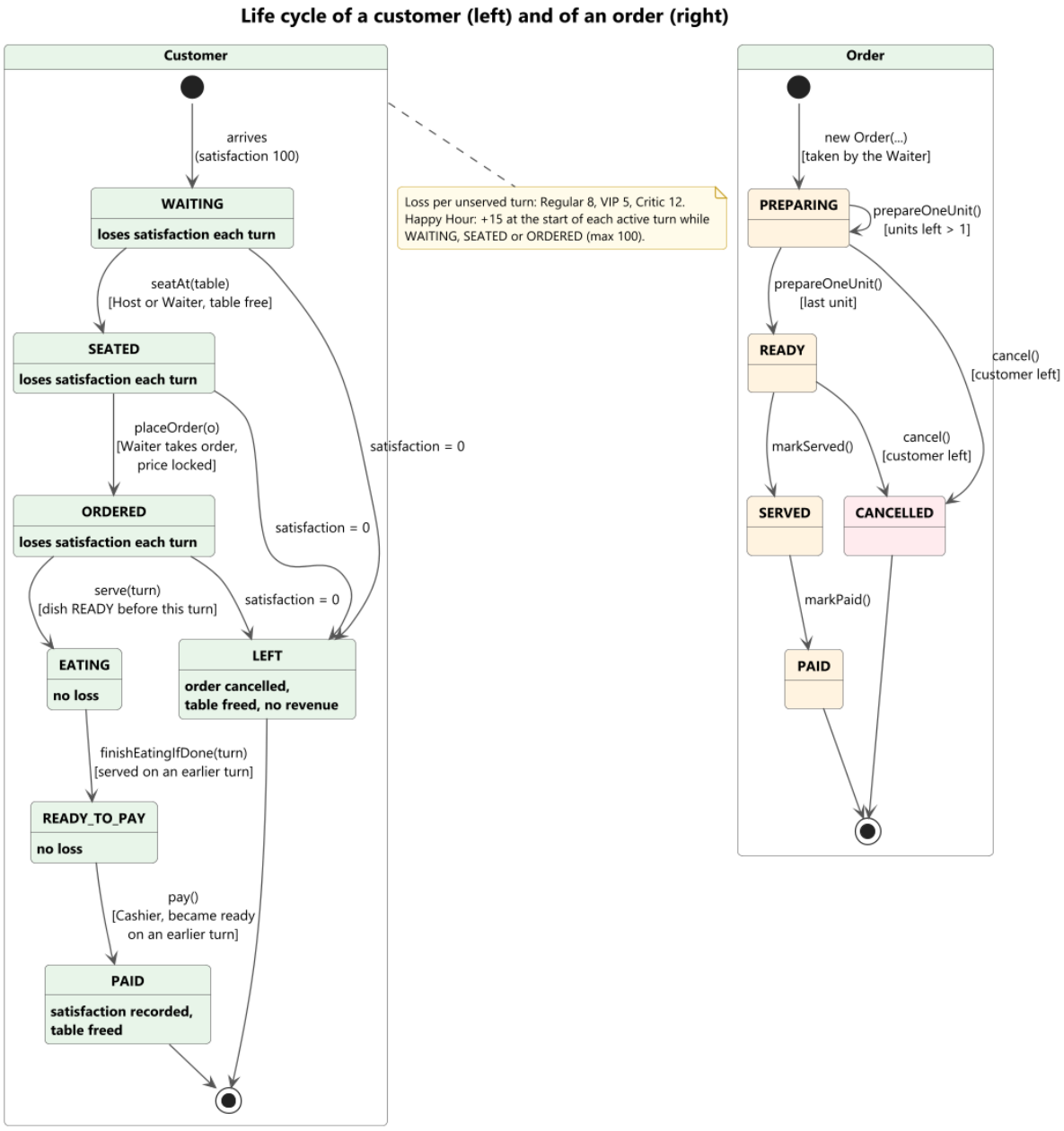


Figure 5. Customer and order life cycles.

4. Design Considerations

4.1 Key abstractions and responsibilities

Class	Responsibility
GameController	Runs the turn steps in order for 18 turns (labelled by the `TurnStep` enum). Hands each manager command to the staff member through `carryOut()` and asks again when a command is rejected. Holds no game rules itself.
GameConfig / CustomerFactory	Base setting: turns, tables, the arrival schedule (turn → `CustomerFactory`, e.g. `VIPCustomer::new`) and the victory targets.
GameResult	Judges the final restaurant state against the three targets.
GameView / ConsoleView / CommandParser	Boundary. Shows the state and the numbered task options, and turns typed text into a `StaffCommand` (syntax only, in `CommandParser`). Never decides whether a move is legal.
Restaurant	Holds the tables, waiting queue, all customers and the results (revenue, satisfaction at payment, departures). Shared rules: seating, finding who may pay, end-of-turn patience, Happy Hour recovery.
Customer (+ Regular, VIP, Critic)	Owns its life cycle and satisfaction. Subclasses decide patience loss, billing, food choice and reaction to service.
Staff (+ Host, Waiter, Chef, Cashier)	One task per turn (`perform()`). `carryOut()` is the common entry point; each subclass accepts only its own commands in `handle()` and lists what it can do now in `availableTasks()` as `TaskOptions`.
StaffCommand / TaskOption / ActionResult	Value objects: a requested task, an option shown to the manager, and the success or failure of an action with the reason.
Order	Locked price, preparation progress and legal status changes (PREPARING → READY → SERVED → PAID, or CANCELLED).
Menu, MenuItem, ComboMeal	What can be ordered. A ComboMeal derives its price and units from its two components.
PriceCalculator, HappyHour	The bill at order time: the customer's own price, then Happy Hour. HappyHour tracks its single use, two-turn duration, discount and recovery.

4.2 Design principles reflected in the code

Single Responsibility (SRP). Rules, flow and display are kept apart:

- `ConsoleView` only prints and reads, and `CommandParser` only parses.
- `GameController` only sequences the turn.
- Each rule lives in one place:
  - "can this be served?" in `Order.canBeServed()`;
  - "who pays next?" in `Restaurant.nextCustomerToPay()`;
  - the one-task limit in `Staff.perform()`;
  - pricing in `PriceCalculator`;
  - money formatting and rounding in `Money`.

Open/Closed (OCP). Stage 2 added two customer types without touching `GameController`, `Restaurant`, `Order`, `Chef` or `Cashier`: the new behaviour is in the new subclasses, and the schedule only registers `VIPCustomer::new` and `CriticCustomer::new`. Stage 3 added the Host as a new `Staff` subclass. Because staff are dispatched polymorphically through `carryOut()`, the controller only had to register it in `directedStaff`; it gained no Host-specific method. Combos are a new `MenuItem`

subclass, so orders, the Chef and pricing handle them unchanged. The one place we had to open a stable class was `Customer`, to add the `reactToService()` hook (Section 6).

Liskov Substitution (LSP). Any `Customer` subclass can be used wherever a `Customer` is expected. The subclasses only change values and choices, never the life-cycle rules, because the state-changing methods are not overridden. A `ComboMeal` behaves exactly like a `MenuItem` (positive price, at least one preparation unit). We deliberately did not make `Host` a subclass of `Waiter`: a Host cannot take orders, so it could not stand in for a Waiter. Both extend `Staff` and share the seating rule through `Restaurant.seatCustomer()`.

Dependency Inversion (DIP). `GameController` depends on the `GameView` interface, not on `ConsoleView`, `Scanner` or `System.out`. This let us add the script-replay mode (the same view reads from a file) without touching the controller. It was tested for real by the optional window: `SwingView` implements `GameView`, and the `gui-extra` commit changes no controller, staff or model file. The arrival schedule depends on the `CustomerFactory` abstraction rather than concrete customer classes.

Interface Segregation (ISP). This principle is only weakly reflected. `CustomerFactory` is a minimal one-method interface. `GameView` is a single interface because both user interfaces need every method; we chose not to split it just to tick the principle.

4.3 Trade-offs

- **Inheritance for customer types** keeps each type small and readable and gives clear polymorphism. However, a customer cannot change type at run time, and a new behaviour mix (e.g. a "VIP critic") would need another subclass.
- **Template method in Staff.** `carryOut()` is final, so every role gets the same `wait` handling and one-task rule, and the controller has no switch on roles. The cost is that a role learns about commands it does not support: the default `handle()` rejects them with a message listing what the role can do.
- **Commands as text with customer numbers** (`seat 2`, `cook 3`) make the game fast to play and easy to script for reproducible runs. We compensate for the weaker guidance with numbered options for every staff member, so the player can type `1` instead of remembering the syntax.
- **Rules inside entities** (e.g. `Customer.pay()` checks its own state) mean some validation is repeated: staff check first to give a friendly message, and entities check again and throw if misused. We accepted this duplication because it makes illegal states impossible even if a future caller forgets a check.

4.4 Alternative design considered

We considered the **Strategy pattern** for customer behaviour: a single `Customer` class holding a `PatiencePolicy`, a `BillingPolicy` and a `DishChoicePolicy`. It would allow mixing behaviours and changing them at run time. We chose inheritance because the brief defines exactly three fixed customer types, each with a few small differences. Three subclasses are simpler to read, to draw in UML and to explain, and they still dispatch behaviour through the customer object. If more types or combinations were needed, we would move to strategies.

We also considered the **State pattern** for the customer life cycle (one class per state). With only seven states and simple transitions, an enum plus guarded methods was clearer and kept all life-cycle rules in one file.

For staff, an earlier version had one controller method per role (`hostTask`, `waiterTask`, `chefTask`) with a switch on the command. We replaced it with polymorphic `carryOut()`/`handle()` so that adding the Stage 3 Host needed no new controller method.

4.5 How the player's choices change the result

- **Waiter priority under constraint.** On turn 13 of stage3-victory, the Waiter takes #5's 3-unit combo before serving #4. #4 waits one more turn, but #5 can then pay on turn 18, giving 5 paid customers instead of 4 (Section 3).
- **Kitchen priority.** In stage3-defeat, the Chef idles and then cooks the newer 2-unit combo before the Critic's 3-unit combo. The Critic (-12 per turn) reaches 0 and leaves on turn 15, so only 3 customers pay ($41.40): Defeat. In stage3-victory, the Chef finishes the Critic's combo first and it is served hot.
- **Serving the Critic on time.** In stage2-victory, the Waiter serves the Critic's Pasta the turn after it became READY ("arrived hot"). In stage2-defeat it is never served, and the Critic walks out.
- **Happy Hour timing.** Using it on turns 13-14 (victory run) discounts #5's combo to $19.44 and lifts #4 from 76 to 98 satisfaction. Using it on turn 4 when nobody is waiting (defeat run) only saves the VIP $2.70 and wastes the recovery.
- **Using the Host.** In the victory run the Host seats while the Waiter takes an order in the same turn, which earns $87.84. The defeat run never uses the Host, and the queue backs up.

5. Functional Evidence and Test Notes

All runs use the base setting and can be replayed exactly with the scripts in `runs/` (check out the matching tag first). Each script contains comments explaining the decisions. Every result below was re-checked by building each tag from a clean checkout and replaying both of its scripts.

Run (tag)	Result	Paid	Revenue	Avg satisfaction / notes
stage1-victory (stage-1)	Victory	5	$66.00	79.2. Includes a rejected early "serve 3" on turn 9 (Figure 7).
stage1-defeat (stage-1)	Defeat	3	$42.00	38.7. Chef idle for many turns; "seat" rejected when no table is free.
stage2-victory (stage-2)	Victory	5	$64.50	78.0. VIP Burger $13.50; Critic served hot.
stage2-defeat (stage-2)	Defeat	3	$31.50	86.0. Critic left unserved and walks out (unhappy departure).
stage3-victory (stage-3)	Victory	5	$87.84	86.6. Host and Waiter in the same turn; Happy Hour turns 13-14; combos.
stage3-defeat (stage-3)	Defeat	3	$41.40	85.0. VIP + Happy Hour Burger $10.80; Critic walks out; cooking the CANCELLED order is rejected.

5.1 Rule checks

Rule from the brief	Where it is shown
Worked example (Pasta: 92, 84, 76, served, pays at 76)	stage1-victory, customer #3: seated at 92, then 84, then 76, served on turn 10, READY_TO_PAY on turn 11, paid on turn 12 with 76.
A dish finished this turn is served next turn at the earliest	`Order.canBeServed()` requires readyTurn < turn; stage1-victory turn 9 "serve 3" is rejected.
READY_TO_PAY this turn pays next turn at the earliest; one payment per turn, oldest first	`Customer.canPay()`, `Restaurant.nextCustomerToPay()`, `Cashier.collectNextPayment()`; stage3-victory #4 and #5 pay on turns 16 and 18.
Invalid choice explained, task not used up	`Staff.perform()` only marks the task used on success; the console shows "Choose again - the Waiter's task has not been used up".
Customer at 0 leaves: order cancelled, table freed, no revenue	stage2-defeat and stage3-defeat (Critic). Cooking the cancelled order is then rejected.
Arrival on turn 16 cannot pay	Customer #6 never pays in any run (at most 5 payers).
VIP, then Happy Hour, price locked	stage3-defeat: Burger $15 x 0.9 x 0.8 = $10.80.
Combo price and units	Salad + Pasta Combo = $24.30, 3 units; Salad + Burger = $21.60, 2; Burger + Pasta = $29.70, 3.
Average satisfaction N/A before anyone pays	Status and end-of-turn lines show "N/A" until the first payment.
Robust input	Unknown commands, missing or non-numeric numbers, option numbers out of range, blank lines and end of input are handled (end of input = wait).

5.2 Console screenshots

The terminal is organised by the turn steps of the brief. Each turn shows:

- a turn header with the turns left;

- a status board (queue, tables, orders, patience and each staff member's available tasks);
- one heading per step;
- numbered options for every choice;
- an end-of-turn summary with each customer's change in satisfaction.

The colours below are added for the report only: events are green, rejected choices red and Happy Hour yellow.

```
HOW A TURN WORKS
  Step 1    Arrivals: the scheduled customer joins the waiting queue.
            Happy Hour can then be switched on (once per game).
  Step 2-3  Your choices: the Host may seat ONE customer, the Waiter does ONE
            task (seat, take an order or serve), then the Chef does ONE
            preparation unit.
  Step 4    Customers served on an earlier turn finish eating; the Cashier
            charges one READY_TO_PAY customer (oldest first).
  Step 5    Every customer not yet served loses satisfaction. A customer at
            0 leaves without paying and frees the table.
  Timing:   a dish finished this turn can be served next turn at the
            earliest; a customer pays the turn after finishing eating.

HOW TO CHOOSE (n = customer number; an order has its customer's number)
  Type the option number shown in the list, e.g.  1
  or type a command:
    Host   : seat n | wait   (acts before the Waiter)
    Waiter : seat n | order n | serve n | wait
    Chef   : cook n | wait
    Happy Hour: answer y or n when asked at the start of a turn
  Short forms: s = seat, o = order, v = serve, c = cook, w = wait.
  When 'wait' is the only option, just press Enter.
```

Figure 6. Welcome screen: how a turn works and how to choose (stage3-victory).

```
TURN 13 of 18                                          5 turns left after this
================================================================================

STEP 1  ARRIVALS
  >> Customer #5 Regular arrives and joins the queue (satisfaction 100).

--------------------------------------------------------------------------------
RESTAURANT AT THE START OF TURN 13
  Revenue $46.80    Paid customers 3    Avg satisfaction (paid) 83.7
  Next arrival: turn 16 (Regular)

  WAITING QUEUE
    #5 Regular   sat 100   leaves in 13 turns if not served
  TABLES
    Table 1   free
    Table 2   #4 Regular   sat  76   ORDERED
              Order: Salad + Burger Combo $21.60 [READY]
              leaves in 10 turns if not served
  STAFF TASKS AVAILABLE
    Host     seat 5 | wait
    Waiter   seat 5 | serve 4 | wait
    Chef     wait
    Cashier  (automatic) nobody ready to pay
--------------------------------------------------------------------------------

  Switch on Happy Hour now? It lasts this turn and next (y/n, Enter = no) > y
  >> Happy Hour switched on: 20% off orders taken this turn and next.
  ** Happy Hour ACTIVE - 2 turns remaining (including this one) **
  >> Happy Hour: #4 Regular recovers to satisfaction 91.
  >> Happy Hour: #5 Regular recovers to satisfaction 100.

STEP 2-3  MANAGER CHOICES, KITCHEN AND SERVING
```

Figure 7a. Start-of-turn board on turn 13 of stage3-victory (the sequence diagram's starting conditions).

```
  ** Happy Hour ACTIVE - 2 turns remaining (including this one) **
  >> Happy Hour: #4 Regular recovers to satisfaction 91.
  >> Happy Hour: #5 Regular recovers to satisfaction 100.

STEP 2-3  MANAGER CHOICES, KITCHEN AND SERVING

  HOST - choose one task:
    1) seat 5   Seat #5 Regular at a free table
    2) wait     Do nothing this turn
  Host > seat 5
  >> Customer #5 Regular is seated at Table 1.

  WAITER - choose one task:
    1) order 5   Take #5 Regular's order
    2) serve 4   Serve Salad + Burger Combo to #4 Regular
    3) wait      Do nothing this turn
  Waiter > order 5
  >> Customer #5 Regular orders Salad + Pasta Combo ($24.30, 3 units). Bill
     locked at $19.44.

  CHEF - choose one task:
    1) cook 5   Cook Salad + Pasta Combo for #5 Regular (PREPARING 0/3)
    2) wait     Do nothing this turn
  Chef > cook 5
  >> Chef worked on Salad + Pasta Combo for order #5: PREPARING 1/3.

STEP 4  EATING AND PAYMENT
  (nobody finished eating or paid this turn)

STEP 5  WAITING AND ABANDONMENT
  (nobody walked out)

------------------------------------------------------------------------------
END OF TURN 13 - RESULTS
  Served 3    Unhappy departures 0    Turns used 13/18
  Revenue $46.80    Avg satisfaction (paid) 83.7
  Customers still in the restaurant:
    #4 Regular   ORDERED       sat  83   (+7 this turn)
    #5 Regular   ORDERED       sat  92   (-8 this turn)
------------------------------------------------------------------------------
```

Figure 7b. Turn 13 choices: Happy Hour, Host seats #5, Waiter takes #5's order at $19.44 instead of serving #4, Chef cooks.

```
  STAFF TASKS AVAILABLE
    Waiter   wait
    Chef     cook 3 | wait
    Cashier  (automatic) nobody ready to pay
------------------------------------------------------------------------------

STEP 2-3  MANAGER CHOICES, KITCHEN AND SERVING

  WAITER - choose one task:
    1) wait   Do nothing this turn
  Waiter (Enter = wait) > serve 3
  !! Not allowed: Order #3 is not READY yet (PREPARING 1/2).
     Choose again - the Waiter's task has not been used up.

  WAITER - choose one task:
    1) wait   Do nothing this turn
  Waiter (Enter = wait) > wait
  >> Waiter waits this turn.
```

Figure 8. A rejected choice is explained and the task is not used up (stage1-victory, turn 9).

```
   >> Customer #2 VIP orders Burger ($15.00, 1 unit). Bill locked at $13.50.
   ...
   >> Customer #3 Critic orders Pasta ($18.00, 2 units). Bill locked at
      $18.00.
   ...
   >> Cashier collected $13.50 from #2 VIP. PAID with satisfaction 90; table
      freed.
   ...
   >> Served Pasta to #3 Critic. The Critic approves: the dish arrived hot.
```

Figure 9. Stage 2 behaviours: VIP discount ($13.50) and the Critic's reaction to a hot dish (stage2-victory).

```
   CHEF - choose one task:
     1) cook 3    Cook Salad + Pasta Combo for #3 Critic (PREPARING 2/3)
     2) wait      Do nothing this turn
   Chef > cook 3
   >> Chef finished Salad + Pasta Combo for order #3: READY (can be served
      next turn).

 STEP 4  EATING AND PAYMENT
   >> Cashier collected $21.60 from #4 Regular. PAID with satisfaction 76;
      table freed.

 STEP 5  WAITING AND ABANDONMENT
   >> Customer #3 Critic ran out of patience and LEFT without paying.
```

Figure 10. The Critic runs out of patience and leaves without paying (stage3-defeat, turn 15).

```
 STEP 4  EATING AND PAYMENT
   >> Cashier collected $19.44 from #5 Regular. PAID with satisfaction 84;
      table freed.

 STEP 5  WAITING AND ABANDONMENT
   (nobody walked out)

 ------------------------------------------------------------------------------
 END OF TURN 18 - RESULTS
   Served 5    Unhappy departures 0    Turns used 18/18
   Revenue $87.84    Avg satisfaction (paid) 86.6
   Customers still in the restaurant:
     #6 Regular   ORDERED       sat  76   (-8 this turn)
 ------------------------------------------------------------------------------


==============================================================================
                              V I C T O R Y !
==============================================================================
   Target                      Result     Needed
   Paid customers              5          >= 4         MET
   Revenue                     $87.84     >= $45.00    MET
   Avg satisfaction (paid)     86.6       >= 60        MET
 ------------------------------------------------------------------------------
   Dishes served 5    Unhappy departures 0    Turns played 18/18
   All three targets met - well managed!
==============================================================================
```

Figure 11. Turn 18 payment and the VICTORY screen (stage3-victory).

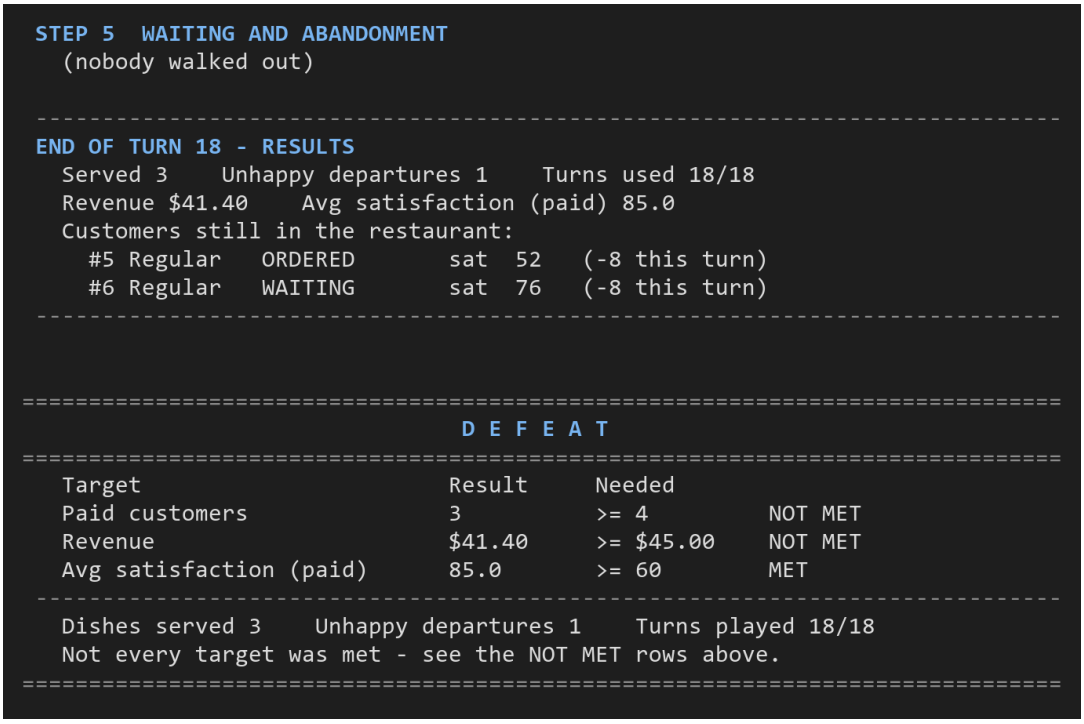


Figure 12. The DEFEAT screen with the targets that were not met (stage3-defeat).

6. Development Process and Stage Changes

Each stage is one commit on the tagged history, and each extends the saved previous stage. The source code itself shows the real stages: for example, `stage-1` contains no VIP, Critic, Host, Happy Hour or combo code, and `stage-2` contains no Stage 3 class.

Tag	Commit	Contents
stage-1	f461480	Regular customers, menu, Waiter, Chef, automatic Cashier, 2 tables, queue, full turn sequence, victory and defeat, two replay scripts.
stage-2	9ce66f8	+ VIPCustomer, CriticCustomer, two replay scripts.
stage-3	87683ee	+ Host, ComboMeal, HappyHour, PriceCalculator, two replay scripts.
gui-extra	d9672e8	+ SwingView (optional window). No rule changes.

Compare links (README): `https://github.com/pyizaw/NTU_SCE1_SC2002/compare/stage-1...stage-2` and `.../compare/stage-2...stage-3`.

6.1 Stage 1 → Stage 2: 6 source files, +143 / -6 lines

File	Change	Reason and effect on the design
customer/VIPCustomer	added	Overrides `getSatisfactionLossPerTurn()` (5) and `billFor()` (10% off).
customer/CriticCustomer	added	Loss 12; overrides `chooseDish()` (most demanding item) and `reactToService()` (cold-food penalty).
customer/Customer	modified	Gained the protected hook `reactToService()` (default: no reaction), and `serve()` now returns the reaction text. Without the hook, the Waiter would need to check "is this a Critic?" to apply the cold-food rule, which the brief forbids.
staff/Waiter	modified	Adds the reaction to the "Served ..." message (3 lines).
controller/GameConfig	modified	Registration only: turn 4 → `VIPCustomer::new`, turn 7 → `CriticCustomer::new`.
view/ConsoleView	modified	The welcome screen explains the customer types.
(deleted)	none	The new types plug into the existing hierarchy.

What still works: `GameController`, `Restaurant`, `Order`, `Chef` and `Cashier` are unchanged, which shows that the Stage 1 design was open to new customer types. The turn order, timing and payment rules are the same, and the Regular customers in the Stage 2 runs follow the same 92 / 84 / 76 patience pattern as in Stage 1.

6.2 Stage 2 → Stage 3: 11 source files, +353 / -8 lines

File	Change	Reason and effect on the design
staff/Host	added	A new `Staff` subclass: `handle()` accepts `seat`, and `availableTasks()` offers the queued customers.
model/ComboMeal	added	A `MenuItem` composed of two `MenuItems`; Order, Chef and pricing use it unchanged.
pricing/HappyHour	added	Single use, two-turn duration, 20% discount and the recovery rule.

File	Change	Reason and effect on the design
pricing/PriceCalculator	added	Customer price (VIP discount) then Happy Hour, so neither class knows about the other.
controller/GameController	modified (+31)	Registers the Host in `directedStaff`, creates the HappyHour and PriceCalculator, and offers Happy Hour at the start of each turn. No new per-role method, because staff are dispatched polymorphically.
staff/Waiter	modified	Asks `PriceCalculator.priceFor()` for the bill instead of `customer.billFor()`, so Happy Hour is applied after the customer's own discount and locked into the order.
model/Menu	modified	`Menu.withCombos()` adds the three combos after the dishes.
model/Restaurant, customer/Customer	modified	`recoverWaitingCustomers()` and `Customer.recover()` implement the Happy Hour recovery; only customers who are still waiting recover.
view/GameView, view/ConsoleView	modified	Happy Hour prompt and status line, and the combo menu on the welcome screen.
(deleted)	none	-

What still works: the six-step turn order, the VIP and Critic rules, Chef cooking, Cashier payment and the result check are unchanged. The Stage 3 runs reuse them, for example the VIP discount applied before Happy Hour ($10.80) and the Critic walking out in stage3-defeat. The Stage 1 and Stage 2 scripts still give the same results at their own tags.

6.3 Optional window version (tag gui-extra)

A GUI earns no marks by itself, so it was added as an extra commit after `stage-3`, and the terminal game remains the main interface. Its value for the design is evidence of DIP:

- `SwingView` is a second implementation of `GameView`.
- The commit only adds `SwingView` and changes `Main` (the `--gui` option). No controller, staff or model file changes.
- Playing the Stage 3 victory script through the window gives the same result: $87.84, 5 paid, average 86.6.

Usability features of the window:

- progress towards each target, shown in green once met;
- an arrival preview;
- numbered task buttons (keys 1-9);
- automatic skipping of staff whose only option is "wait";
- a "leaves in N turns" warning on every waiting customer, computed by the domain query `Customer.turnsUntilLeaving()`.

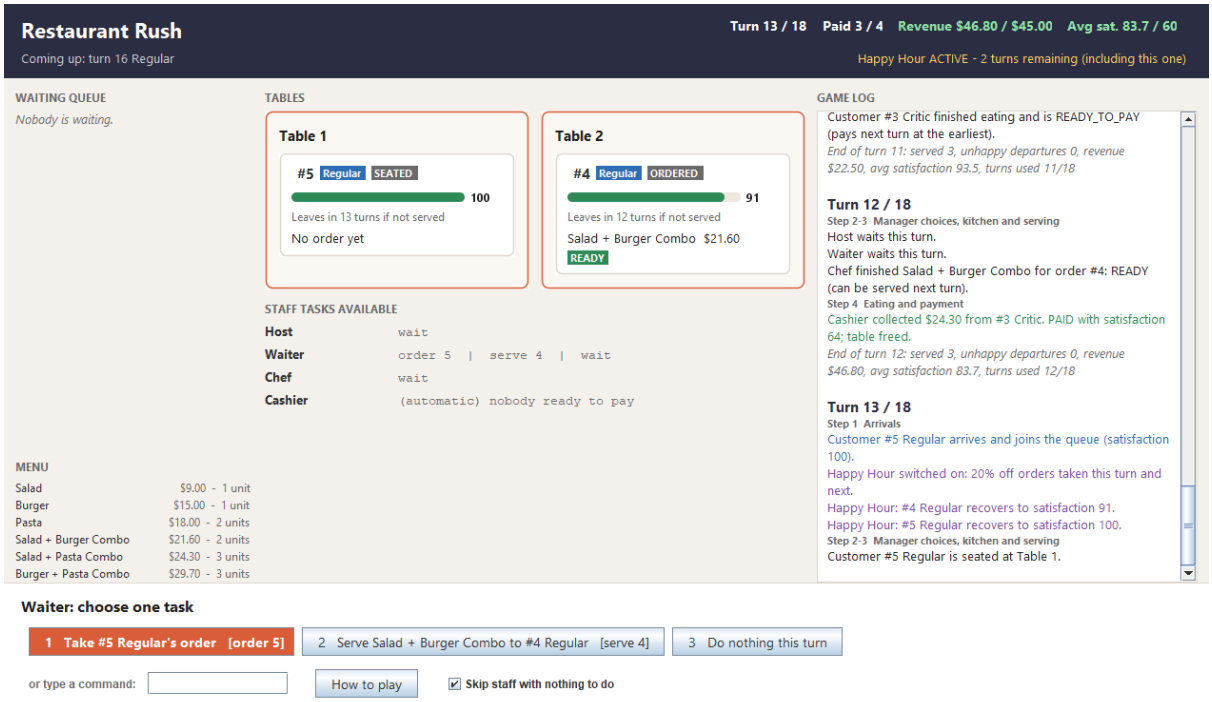


Figure 13. Window version, turn 13 of the Stage 3 victory run.

7. Reflections

Group reflection.

- **What went well:** keeping the rules inside the domain objects meant each stage was mostly additive. The deterministic food rule and the script-replay mode made it easy to reproduce runs and to check the numbers by hand.
- **What we would improve:** a unit-test suite (e.g. JUnit) for the pricing and timing rules, a configurable difficulty setting, and possibly strategies instead of subclasses if more customer types were added.

Each member answers the four questions below under their own name, using examples from their own work (classes, commits or diagrams).

7.1 [Member 1 name]

Contribution: [the stages, classes, diagrams or report sections you worked on]

What did you learn?
[Describe a specific gain in knowledge or skill.]

How did you apply it?
[Connect a course concept to a real project decision.]

What challenged you?
[Explain the hardest part and why it was difficult.]

How did you solve it?
[Describe your attempts, your solution and what you learned.]

7.2 [Member 2 name]

Contribution: [the stages, classes, diagrams or report sections you worked on]

What did you learn?
[Describe a specific gain in knowledge or skill.]

How did you apply it?
[Connect a course concept to a real project decision.]

What challenged you?
[Explain the hardest part and why it was difficult.]

How did you solve it?
[Describe your attempts, your solution and what you learned.]

7.3 [Member 3 name]

Contribution: [the stages, classes, diagrams or report sections you worked on]

What did you learn?
[Describe a specific gain in knowledge or skill.]

How did you apply it?
[Connect a course concept to a real project decision.]

What challenged you?
[Explain the hardest part and why it was difficult.]

How did you solve it?
[Describe your attempts, your solution and what you learned.]

7.4 [Member 4 name]

Contribution: [the stages, classes, diagrams or report sections you worked on]

What did you learn?
[Describe a specific gain in knowledge or skill.]

How did you apply it?
[Connect a course concept to a real project decision.]

What challenged you?
[Explain the hardest part and why it was difficult.]

How did you solve it?
[Describe your attempts, your solution and what you learned.]

7.5 [Member 5 name]

Contribution: [the stages, classes, diagrams or report sections you worked on]

What did you learn?
[Describe a specific gain in knowledge or skill.]

How did you apply it?
[Connect a course concept to a real project decision.]

What challenged you?
[Explain the hardest part and why it was difficult.]

How did you solve it?
[Describe your attempts, your solution and what you learned.]

8. Declarations and AI History

The signed **Declaration of Original Work** and each member's **Declaration on Use of GAI Assistance** (course forms) are included in the submission archive.

[Attach the signed declarations.]

AI assistance was used in this project and is declared on the GAI forms. As the brief requires, each member's full AI conversation history (prompts, replies and follow-ups, in order) is on their own branch of the group repository, in `ai-history/stage-1/chat-01.md`, `ai-history/stage-2/chat-01.md` and `ai-history/stage-3/chat-01.md`. The README links to each member's branch.

Member	AI history branch
[Member 1 name]	[branch link]
[Member 2 name]	[branch link]
[Member 3 name]	[branch link]
[Member 4 name]	[branch link]
[Member 5 name]	[branch link]